

0.19.2. Revenir : un pas négatif — doublé, deux positions

L'aller-retour du A (ligne 0), puis celui du B (ligne 8), chacun avec sa variable.

1. Ce qu'il faut comprendre

C'est le 0.19 et le 0.19.1 réunis : les deux allers-retours, chacun avec sa variable (x, xb).

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1664 octets** de cartouche.

1. UNE VARIABLE — PAS

```
uint8_t pas = 5;
```

2. UNE VARIABLE — X

```
uint8_t x = 0;           // le A
```

3. UNE VARIABLE — XB

```
uint8_t xb = 0;          // le B
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    x = deplace_x(x, 0, ALPHABET[0], pas);    // le A : aller...
    x = deplace_x(x, 0, ALPHABET[0], -pas);    // ...et retour (le 0.19)
    xb = deplace_x(xb, 8, ALPHABET[1], pas);    // le B : aller...
    xb = deplace_x(xb, 8, ALPHABET[1], -pas);    // ...et retour (le 0.19.1)

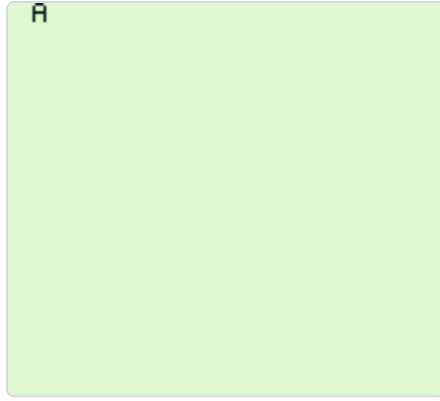
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

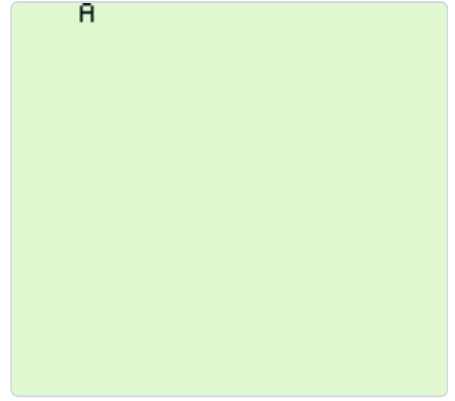
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



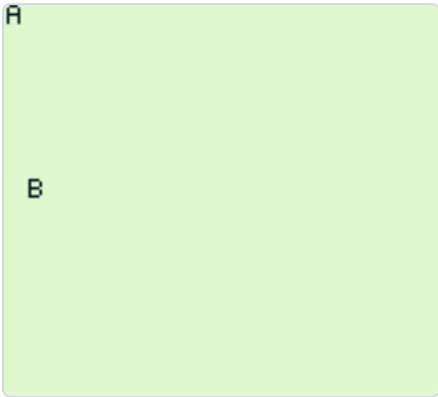
la toute première image — la console vient de démarrer — image 1



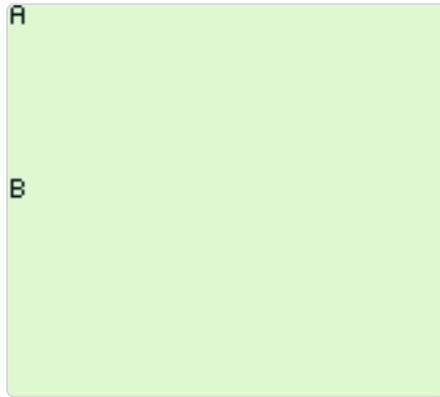
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Le A va et revient sur la ligne 0, puis le B sur la ligne 8.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A passe par (5, 0)
- ✓ le A finit en (0, 0) (0,0)
- ✓ le B passe par (5, 8)
- ✓ le B finit en (0, 8) (0,8)

Ensuite : **0.20. Voir x à l'écran : nombre** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++